CITS5501 Project Phase 2 – Group 16

October 2026

1. Introduction

This report covers Phase 2 of the CITS5501/CITS3501 project, in which we build a minimal scraper for the UWA Policy Library (https://uwa-policy.cits5501.arranstewart.io/policy/) and plan how a production version of it should be tested. It addresses the four Phase 2 tasks:

- **Section 3 (Task 1)** describes our minimal Python prototype, which identifies policy-document links on the index page, downloads five distinct documents and prints their metadata, together with its assumptions and limitations.
- **Section 4 (Task 2)** proposes nine test cases covering the five failure scenarios given in the brief.
- **Section 5 (Task 3)** discusses how fixtures and test doubles make those tests repeatable, reports our observations of the site's request limits, and explains our sample artifact: a local mock server that returns HTML for a PDF link.
- **Section 6 (Task 4)** lists seven adversarial test ideas that challenge assumptions a scraper commonly makes about its environment.

The prototype code and its README (`Task1/`) and the Task 3 fixture code (`Task3/`) are included in the accompanying zip file.

Our group's repository is hosted at: github.com/TysonHaines/CITS5501-Group-16.

2. Team members

Group name: 404 Brain Not Found (Group 16).

Name	Student number	GitHub username
Tyson Haines	23779585	TysonHaines
Akshay Raghuwanshi	24991278	AkshayRaghuwanshi
Khushi Sesetti	23823678	khushisesetti
Kevin Tan	23789621	phattydumpling
Celine Xu	24714078	1031Celinela

2.1. Task allocation

Task	Contributors
Task 1 – Prototype	Tyson/Khushi/Kevin
Task 2 – Test suite design	Celine/Kevin
Task 3 – Fixtures and test doubles	Akshay

Task	Contributors
Task 4 – Adversarial inputs	Khushi
Report integration/Final Review	Kevin/All Members

3. Task 1 – Prototype

3.1. Purpose and Implementation

Our minimal prototype identifies and attempts to download a small selection of policy documents linked from the supplied UWA Policy Library index page.

In simple terms, it works by retrieving all hyper reference links starting from the index page. It then filters through these links by identifying which links contain a file extension `e.g. '.pdf', '.doc', etc.`, where those that do not contain any of the expected extensions are discarded. From there five 'distinct' policies are selected. The program will attempt to 'download' these policies via a stream (nothing is saved to disk as per Phase 2 brief requirement 3.2.3). The program will then display the metadata for this policy download request; including key information used to determine the outcome:

- URL of the policy document
- HTTP status and reason
- Declared `Content-Type`
- Downloaded size
- First 16 bytes of the file
- Outcome of the download: `SUCCESS` or `FAILED`

3.2. Request Controls

All HTTP requests initiated explicitly by `scraper.py` pass through a common `fetch()` function.

This applies:

- A group-specific User-Agent identifying the course project.
- A 10-second connection/read timeout.
- A one-second pause between consecutive calls.
- A limit of 20 calls to request wrapper per run.

With the default selection of five documents, the normal workflow requires one index fetch and up to five document fetches. The prototype does not retry failed requests or select replacement documents when a selected download fails.

These controls reduce the load placed on the website but they have limits. The timeout does not impose a ten-second deadline in the entire download. In addition, redirects followed automatically by requests can generate additional HTTP requests within a single `fetch()`. Those redirect requests are not individually counted or delayed by the wrapper.

3.3. Assumptions

A1 - Directly available links. Policy documents are accessible through links in the index page's initial HTML response. Recursive crawling and JavaScript execution are outside the prototype's scope.

A2 - Extension-based identification. A recognised URL extension is sufficient to identify a potential policy document. This is a discovery heuristic, not proof that the response contains a usable document.

A3 - URL-based document distinctness. Resolved URLs are treated as distinct after removing fragments. The prototype does not compare document contents to identify identical files served under different URLs.

A4 - Small demonstration sample. Selecting the first five candidate URLs is sufficient to demonstrate discovery and downloading; the prototype is not intended to retrieve the entire policy collection.

A5 - Execution environment. The prototype is run on a machine with `python3` (version 3.10 or later, as required by the pinned `requests` library) and `uv` installed, as specified for the marking environment (Ubuntu 22.04). `uv` installs the pinned dependencies listed in `Task1/requirements.txt`. Where `uv` is unavailable, a standard Python virtual environment and `pip` can be used instead, as described in `Task1/README.md`.

3.4. Limitations

L1 - Incomplete document discovery. Extension filtering can miss documents served through extension-less endpoints, viewer pages or unsupported extensions such as `.rtf` and `.xlsx`. It can also select a URL ending in .pdf even when the response is not a policy document. Links generated by JavaScript are not discovered because the scraper only parses the returned HTML. The filter also does not check which website a link points to, so documents hosted elsewhere (the index links to several on www.uwa.edu.au) are also candidates. The brief does not require such links to be included; on the live index none fall within the first five selected documents, so in practice the prototype only downloads from the policy site.

L2 - No document validation. The prototype checks HTTP status and prints the declared content type and initial bytes, but does not validate PDF or office-file structure. Consequently, an HTTP 200 response containing an HTML error page or a corrupt PDF can still be reported as a successful download. 'SUCCESS' indicates a successful HTTP response whose body was read without a reported error, not proof that the document is usable. Tasks 2 and 4 propose tests that expose this distinction.

L3 - No download-size limit. The prototype reads documents in chunks without saving them to disk or loading the complete body into memory. However, it does not enforce a maximum download size. A very large response could therefore consume substantial bandwidth and processing time.

L4 - Limited failure handling and network controls. The prototype has no retry policy or overall download deadline. Its connection/read timeout does not guarantee that a continuously

arriving response will finish within ten seconds. Index-fetch errors are not caught and an HTTP error response from the index can be parsed as though it were a normal page. Additionally, automatic redirects are not individually counted or delayed by `fetch()`, so its counter does not enforce a strict cap on all actual HTTP requests.

4. Task 2 – Design of Test Suite

4.1. Scope and expected behaviour

We propose nine black-box tests for a **fully functional policy-document scraper**, rather than for the minimal Task 1 prototype. The aim is predictable behaviour under faulty website structures, incomplete responses, network failures, HTTP errors and invalid documents. Correct behaviour may mean reporting a failure; recovery is not always possible. A failed document should not prevent other valid documents from being processed, whereas failure to retrieve the index must not be reported as a successful run with zero documents.

Our proposed acceptance policy is to use a configurable timeout and **at most two attempts per URL** for timeouts or HTTP 503, with **no automatic retry for HTTP 404**, subject to an overall cap of **20 HTTP requests**. These are explicit test-design assumptions, **not claims about the current prototype**. A downloaded document is regarded as valid only if its content is consistent with the expected format and can be parsed; an HTTP 200 response alone is insufficient.

The reasonable response for each specified scenario is:

1. **Changed website structure:** Extract correct document links if supported; otherwise report that document discovery failed instead of silently claiming success.
2. **Truncated HTML:** Process only reliably recoverable links without crashing or inventing URLs. Detect and report an incomplete HTTP transfer where it is observable; malformed markup alone does not always prove truncation.
3. **Short or long timeout:** Allow bounded recovery from a temporary failure; report a persistent timeout and stop retrying within the configured limits.
4. **HTTP 4xx or 5xx:** Record the actual error status; avoid retrying permanent 4xx responses (such as 404), bound retries for 5xx, and continue with independent links where feasible. A 429 rate-limit response may require separate handling of `Retry-After`.
5. **Invalid policy document:** Distinguish a successful HTTP transfer from a usable PDF/office document; reject wrong-content and corrupt files with an explicit reason.

4.2. Test cases

Each test uses a controlled local index (e.g., `http://localhost:3000/policy/`) with predetermined responses, or a test double that supplies the stated failure. Valid PDF fixtures are independently confirmed to open in a reference PDF reader; the corrupt PDF fixture is independently confirmed to be rejected. **Act** for every case is to run the production scraper against the test index, except where the index fetch itself is being simulated. Assertions are checked using the reported outcomes, discovered URLs, request log and, where applicable, document-validation results. No live-site traffic is required.

Test ID / scenario	Arrange (fixture or environment)	Act	Assert (pass/fail evidence)
T001 - S1: Changed page structure	Return HTTP 200 with an index whose policy URLs appear in `<button data-download="/docs/a.pdf">` rather than the expected `<a href>` elements. Include a valid `/docs/a.pdf` endpoint.	Scrape the altered index.	**Pass:** either discover and validate `a.pdf` using a supported fallback, **or** explicitly report document-discovery failure. **Fail:** quietly report successful completion with zero policies, invent links, or crash unexpectedly.
T002 - S2: Truncated HTML markup	Return HTTP 200 with a complete `<a href="/docs/good.pdf">Good</a>` followed by a second, unfinished tag (`<a href="`) at end of body. `/docs/good.pdf` is valid.	Parse the incomplete index and attempt downloads.	**Pass:** recognise the complete `good.pdf` link, do not request a fabricated URL from the unfinished tag, and do not crash. **Fail:** attempt to download an incomplete/invalid link or treat malformed markup as a valid document link.
T003 - S2: Incomplete HTTP response	The index endpoint announces `Content-Length: 1000` but sends only 200 bytes before closing the connection.	Request the index.	**Pass:** report an incomplete/failed transfer and do not treat the partial index as fully retrieved; any retries remain bounded. **Fail:** report successful index retrieval or continue as though the full body arrived.
T004 - S3: Temporary document timeout	The index links to `/docs/recover.pdf`. A response double raises a timeout on its first GET, then returns HTTP 200 with a valid PDF on its second GET.	Download `recover.pdf`.	**Pass:** exactly two attempts, one successfully validated document, no duplicate result and no exceeded request cap. **Fail:** give up before the permitted retry, loop indefinitely or count the file twice.
T005 - S3: Persistent index timeout	The index request times out on every attempt; the double records how many GETs were made.	Start a scraping run.	**Pass:** stop after at most two attempts and report that the index could not be retrieved. **Fail:** hang/retry without

Test ID / scenario	Arrange (fixture or environment)	Act	Assert (pass/fail evidence)
			bound or report a successful empty scrape.
T006 - S4: HTTP 404 on one link	A valid index links to `/docs/missing.pdf` (HTTP 404) and `/docs/good.pdf` (HTTP 200, valid PDF).	Process both links.	**Pass:** record `missing.pdf` as unavailable, make only one request to it, and still process `good.pdf` successfully. **Fail:** label the 404 body as a downloaded policy or abandon the entire run unnecessarily.
T007 - S4: Persistent HTTP 503	The index links to `/docs/busy.pdf` (returns HTTP 503 on both attempts) and `/docs/good.pdf` (valid HTTP 200 PDF). Record all requests.	Process both links.	**Pass:** at most two requests to `busy.pdf`, report HTTP 503 as a document failure, and process `good.pdf`. **Fail:** retry indefinitely, accept the error body as a document or hide the error.
T008 - S5: HTML returned for a PDF URL	`/docs/policy.pdf` returns HTTP 200 with `Content-Type: text/html` and an HTML error page, despite the `.pdf` URL suffix.	Download and validate the response.	**Pass:** reject it as a PDF and report a content/format mismatch. **Fail:** classify it as a usable PDF merely because the status is 200 or the URL ends in `.pdf`.
T009 - S5: Corrupt PDF with a valid-looking signature	`/docs/broken.pdf` returns HTTP 200, `Content-Type: application/pdf`, and bytes beginning `%PDF-1.7`, but the remaining structure is malformed; a reference PDF parser rejects the fixture.	Download and validate the response.	**Pass:** mark the document invalid/corrupt and do not report successful document validation. **Fail:** accept it as usable solely because of its HTTP status, MIME type or initial `%PDF` signature.

4.3. Test rationale and limitations

The nine cases cover all five required scenarios: **S1 (1 case), S2 (2), S3 (2), S4 (2), S5 (2)**. The paired cases intentionally distinguish different failure mechanisms: syntactically incomplete HTML versus a demonstrably incomplete network transfer (T002–T003), temporary versus persistent timeouts (T004–T005), client versus server HTTP errors (T006–T007), and misleading content

types versus structurally corrupt documents (T008–T009). Observable outcomes, reference fixtures and request counts provide test oracles without depending on exact error-message wording.

The suite also has limitations. Both S4 cases fail individual document links, where neither covers the whole site returning an error, or the HTTP 429 rate limiting with `Retry-After` that we observed on the target site (Section 5.2). In addition, the retry limits and request cap are our own test-design assumptions rather than requirements from the brief, so a scraper with a different but reasonable retry policy would need the expected attempt counts adjusted.

These are **proposed tests, not executed test results**. Their fixtures and response doubles can be implemented and reused independently of the live site. They define desirable production behaviour, and **do not assert that the intentionally minimal Task 1 prototype already passes them**.

5. Task 3 – Fixtures and Test Doubles

5.1. Fixtures as a Source of Truth

Testing a web scraper against a live website can be unreliable because the website's content and behaviour may change over time. To make our tests more repeatable, we can use fixtures containing predefined inputs and expected results.

For HTML parsing, we would maintain sample policy index pages containing a known set of document links. These would allow us to check whether the scraper identifies the correct documents. We could also create modified versions with missing elements, incomplete HTML or changes to the page structure to test how the scraper handles unexpected content.

Similarly, valid and deliberately corrupted PDF and Word documents could be used to check whether downloads are correctly validated. The expected file formats and results would be established beforehand, allowing us to determine whether the scraper is producing correct output rather than simply completing a request.

A locally hosted HTTP server is also useful because it allows us to control the status codes, content types and response bodies returned to the scraper. This makes it possible to reproduce failures consistently without relying on the live website.

5.2. Rate-Limiting Observations

We used `curl` to investigate whether the target policy website applies request limits.

Method: On 15 October 2026, we first accessed the policy index page, which returned HTTP 200. We then made six download requests for the same PDF document, with a one-second delay between requests. For each attempt, we recorded the HTTP status, content type, downloaded size and response time.

Results:

Request	HTTP Status	Content-Type	Size (bytes)	Response Time (s)
1	200	application/pdf	301,201	0.554
2	200	application/pdf	301,201	0.538
3	200	application/pdf	301,201	0.599
4	200	application/pdf	301,201	0.505
5	200	application/pdf	301,201	0.517
6	200	application/pdf	301,201	0.528

All six requests returned HTTP 200 with identical download sizes. Response times remained between approximately 0.50 and 0.60 seconds, and no redirects occurred.

Second test: Since no limit appeared within six requests, on 19 October 2026 we repeated the test with ten download requests for the same PDF, again one second apart, and this time also recorded the response headers (using `curl -D -`).

Request	Time	HTTP Status	Content-Type	Size (bytes)	X-Ratelimit-Remaining	Retry-After (s)
1	19:09:12	200	application/pdf	301,201	2	10
2	19:09:14	200	application/pdf	301,201	2	10
3	19:09:16	200	application/pdf	301,201	1	7
4	19:09:17	200	application/pdf	301,201	1	7
5	19:09:19	200	application/pdf	301,201	0	4
6	19:09:20	200	application/pdf	301,201	0	4
7	19:09:21	**429**	text/html	40,740	0	1
8	19:09:23	**429**	text/html	40,740	0	2
9	19:09:24	200	application/pdf	301,201	2	10
10	19:09:25	200	application/pdf	301,201	2	10

Every response, including the successful ones, also carried the header `X-Ratelimit-Limit: 3`.

Conclusion: The site does limit request rates. Requests 1–6 succeeded, but requests 7 and 8 were refused with HTTP 429 (Too Many Requests) and a 40,740-byte HTML error page in place of the PDF. The `Retry-After` header counted down to zero, and from request 9 the remaining count reset and downloads succeeded again. In practice the site therefore allows about six requests per window of roughly ten seconds. Although the header advertises a limit of 3, six requests succeeded because the remaining count fell by one only every two requests. Our first test made exactly six requests, which fitted within this allowance; that is why it showed no limit. The headers on those successful responses already reported the remaining allowance, but we had not recorded them.

For a larger test suite, we would avoid repeatedly accessing the live website. Instead, we could use local fixtures and simulated server responses to test error handling, retry behaviour and rate limiting without generating unnecessary traffic.

5.3. Using Test Doubles to Simulate Failures

Test doubles allow us to reproduce network failures that may be difficult to trigger consistently against the actual policy website.

In Python, `unittest.mock` or pytest's `monkeypatch` fixture can replace HTTP operations with controlled behaviour. For example, a mocked request could raise a timeout or connection exception, allowing us to check whether the scraper reports the error and handles retries appropriately.

A local HTTP server could also simulate network problems more realistically. Adding a delay before responding would allow us to test slow responses and timeouts. An interrupted download could be reproduced by closing the connection before the complete response body has been sent. Similarly, returning HTTP 404, 429 or 503 would allow us to examine how the scraper handles unavailable documents, request limits and server errors.

These approaches make testing more predictable because the same conditions can be reproduced whenever needed. They also allow us to test different failure scenarios without depending on the availability or behaviour of the live website.

5.4. Implemented Failure Scenario

Scenario: Our implementation demonstrates **T008 (S5 – HTML returned for a PDF URL)** from Task 2.

We created a local HTTP server using Python's `http.server` module. The server simulates a policy website containing a link to `/policy/invalid-policy.pdf`.

Although the requested URL ends in `.pdf`, the server deliberately returns HTTP 200 with `Content-Type: text/html`. The response contains an HTML error message rather than a PDF document.

Why this scenario: We chose this scenario because a scraper that checks only the HTTP status and file extension may incorrectly accept an invalid document. According to T008, a fully functional scraper should reject the response and report a content or format mismatch.

Files:

- `Task3/mock_server.py` – Implements the local HTTP server and invalid PDF response.
- `Task3/test_mock_server.py` – Contains three automated tests using pytest.
- `Task3/README.md` – Provides instructions for running the server and reproducing the demonstration.

How to run:

These instructions assume that `python3` (version 3.10 or later) and `uv` are installed, as specified for the marking environment (Ubuntu 22.04). `uv` is used to create the virtual environment and install pytest from `Task3/requirements.txt`, as described in `Task3/README.md`. The mock server itself uses only Python's standard library.

From the repository root, activate the Python environment and execute:

```
python -m pytest -v Task3/test_mock_server.py
```

To run the mock server manually:

```
python Task3/mock_server.py
```

The mock policy index is then available at `http://127.0.0.1:8000/policy/`.

Test results: All three automated tests passed locally using Python 3.10.12 and pytest 9.1.1 on WSL Ubuntu 22.04.

The tests verified that the index contained the expected PDF link, that requesting the document returned HTML instead of PDF content, and that a simplified scraper incorrectly accepted the response.

We also tested the group's actual Task 1 prototype against the mock server. The scraper discovered the document link and downloaded the response. However, despite receiving 234 bytes of HTML with the content type `text/html`, it reported:

```
HTTP status and reason:
200 OK

Content-Type:
text/html

Downloaded Size:
234 (0.00 MB)

Outcome: SUCCESS

Downloads succeeded: 1 of 1 attempted
Requests made: 2 (limit 20)
```

This demonstrated that the prototype currently determines download success using the HTTP status without validating the actual document format.

Under T008′s acceptance criteria, this behaviour would be considered a failure for a production scraper. However, it is a limitation of the current prototype rather than necessarily a violation

of Task 1 requirements, which focus on basic document discovery, downloading and metadata reporting.

Reproducing this scenario against the live website would be difficult because we cannot control its server responses. Using a local test double allowed us to reproduce the issue consistently without interfering with the actual website.

5.5. Limitations

Our implementation focuses on one specific failure scenario: a PDF URL returning HTML content despite a successful HTTP response.

Although the three automated tests confirm that the test double produces the intended behaviour, they do not provide comprehensive testing of the scraper. The integration test demonstrates the prototype's response to this particular scenario, but does not establish how it would behave under other network failures.

The current test double does not simulate connection timeouts, interrupted downloads, or repeated HTTP 429 and 503 responses. These could be added in future tests to evaluate retry behaviour and error handling.

Another limitation is that our demonstration focuses on incorrect content types and file signatures rather than full document parsing. A more complete test suite would also include corrupted PDFs that have valid-looking headers but cannot be opened by a PDF reader, as described in T009.

Despite these limitations, the implemented fixture provides a repeatable demonstration of an important weakness in document validation and can be reused when improving the scraper.

6. Task 4 – Adversarial Inputs

Our phase 1 report identified assumptions about stable website structure, accurate content-type headers and predictable network behaviour. We extend those into adversarial test ideas below. These tests would use controlled local fixtures or servers rather than deliberately disrupting the live website. They are proposed tests, not observed results.

Assumption challenged	Input artifact or environment	Failure and how it would be detected
A document can be trusted when its extension, HTTP status and declared content type look correct.	A local endpoint `/policy.pdf` returns HTTP 200 and `Content-Type: application/pdf`, but its body is an HTML error page.	**Failure:** The scraper reports the document as valid or usable without detecting the mismatch. **Detection:** Compare the scraper's validation result against the known HTML fixture. Reporting a successful HTTP transfer alone is not a failure.

Assumption challenged	Input artifact or environment	Failure and how it would be detected
A PDF signature proves that a file is a usable PDF.	Serve a file beginning with `%PDF-1.7` but containing invalid data and no usable PDF structure. Independently confirm that a reference PDF parser rejects the fixture.	**Failure:** The scraper declares the file valid based only on its initial bytes. **Detection:** Compare the scraper's validation result with the reference parser's rejection of the same file. This exposes the limitations of signature-only validation.
One call to the HTTP wrapper corresponds to one HTTP request.	Configure a document endpoint to return a chain of about 25 HTTP redirects: long enough that following it would require more than 20 HTTP requests in total, including the index request, if the request cap were not enforced, but below the HTTP library's own redirect limit (30 for Python `requests`), so the test is not cut short by the library.	**Failure:** The scraper exceeds the request cap while its internal counter remains below the limit. **Detection:** Count every actual HTTP request, including redirect hops and retries, in the local server log and compare the total with the configured cap and scraper's reported counter.
Resolving relative links against the response URL is always sufficient.	Serve `/index.html` containing `<base href="/documents/">` and `<a href="policy.pdf">`. Configure `/documents/policy.pdf` to return a valid PDF and `/policy.pdf` to return HTTP 404.	**Failure:** The scraper requests `/policy.pdf` rather than `/documents/policy.pdf`. **Detection:** Compare the logged request path with the expected path determined by the HTML base URL.
Every document is small enough to load into memory safely.	A local endpoint streams a large response body without a `Content-Length` header. Run the scraper with a predefined download-size or resource budget.	**Failure:** The scraper exceeds the budget, exhausts memory or terminates unexpectedly instead of reporting an oversized download. **Detection:** Monitor received bytes, peak memory usage and exit status against the predefined limits.
A per-request timeout guarantees a	A local endpoint sends small data chunks at intervals	**Failure:** The scraper continues beyond the overall time budget

Assumption challenged	Input artifact or environment	Failure and how it would be detected
bounded total download time.	shorter than the configured read timeout but continues beyond a predefined overall time budget. For this test, the overall time budget is an additional test assumption.	without stopping or reporting the slow download. **Detection:** Use an independent wall-clock watchdog to measure elapsed time. This tests whether the scraper incorrectly treats a read timeout as a total download deadline.
Different link strings always identify different documents.	Serve an index at `/index.html` containing `/policy.pdf`, `./policy.pdf`, `/policy.pdf#page=1` and repeated copies of the same link. All resolve to the same document endpoint. These URLs refer to the same document resource, with the fragment not affecting the HTTP request.	**Failure:** The scraper downloads or counts the same document multiple times as distinct documents. **Detection:** Compare the local server's request log and the scraper's reported document count with the expected single unique document.